



Universidad de Huelva



Escuela Técnica Superior de Ingeniería
Universidad de Huelva

MASIC: Guía rápida de programación

TRABAJO DE FIN DE GRADO
Grado en Ingeniería Informática

Autor: Diego Cerezo Rojas

12 de enero de 2026

Resumen

En esta pequeña guía, se explica (de manera **muy breve** y para personas con algo de experiencia en ensamblador) qué es todo lo que tenemos que tener en cuenta al escribir un programa para el sistema **MASIC**, las características del lenguaje, el conjunto de instrucciones y lo más fundamental de la arquitectura para comenzar a escribir un programa y probarlo en el emulador, además de hacerlo compatible con el sistema físico real. **Esto no es la memoria del proyecto** ni un anexo de la misma, si no un pequeño documento diseñado como una primera toma de contacto con aquellos ajenos al lenguaje y la arquitectura de MASIC, además de cómo usar el emulador.

Índice general

1. EMASIC: Pequeña guía para el lenguaje	4
1.1. Introducción	4
1.1.1. Breve descripción de la arquitectura	5
1.1.2. El adaptador de vídeo	6
1.2. Directivas	7
1.3. Instrucciones	8
1.3.1. Códigos de operación por instrucción y modo de direccionamiento	9
1.4. Etiquetas y comentarios	11
1.5. Macros	13
1.5.1. Etiquetas locales dentro de una macro	15
1.6. Expresiones	16
2. QTMASIC: Cómo usar el emulador	17
2.1. Cargar código fuente	17
2.2. Cargar código ya ensamblado	17
2.3. El depurador	18
2.3.1. Configuración	18

Índice de cuadros

1.1. Directivas del ensamblador MASIC	7
1.2. Distintos tipos de direccionamiento	8
1.3. Primera tabla de códigos de operación	9
1.4. Segunda tabla de códigos de operación	10

Capítulo 1

EMASIC: Pequeña guía para el lenguaje

1.1. Introducción

EMASIC es un **macroensamblador monolítico y cruzado** para el sistema **MASIC**. El programa toma el código fuente como entrada y produce como salida un archivo en código máquina, el cual no es relocizable. No genera código objeto intermedio ni precisa de un programa cargador para ser ejecutado, todas las direcciones se calculan de manera absoluta y el programa está listo para ser colocado en la memoria FLASH directamente desde la primera dirección de memoria. Un ejemplo de cómo se usa desde la línea de comandos, ya sea en Windows o en Linux:

```
emasic source.masic out.mmc
```

La extensión de los archivos realmente no importa y no se comprueba, pero los archivos de código fuente tendrán por defecto la extensión “**.masic**” y el código máquina que el programa genera en la salida tendrá extensión “**.mmc**” (MASIC Machine Code). Este programa (out.mmc) será colocado precisamente en la FLASH, que comienza en la dirección **0x0000** del espacio de memoria principal, cosa que hará el emulador automáticamente al cargarlo.

1.1.1. Breve descripción de la arquitectura

El procesador de 1 dirección (**basado en acumulador**) tiene tamaño de palabra de **8 bits** y un espacio de direcciones de **16 bits**, con un **mapa de memoria no común**, esto es, 64KiB para la memoria principal y 64KiB para el espacio de E/S. La memoria principal puede accederse a través de las instrucciones **LOAD y STORE**, y el espacio de E/S puede accederse con **IN y OUT**, instrucciones que usan el acumulador como origen/destino.

En memoria principal tenemos una FLASH de 8KiB al comienzo, y los 56KiB restantes son SRAM, así que para escribir un programa hay que tener en cuenta diversas limitaciones:

- Los primeros **8KiB** son de **sólo lectura**. En el emulador esta restricción no existe, pero en hardware real sí, y hay que tenerlo en cuenta a la hora de hacer el programa si quiere que sea 100 % compatible con el hardware físico. Intentar escribir en estas direcciones en la máquina real tendrá un efecto nulo.
- Los primeros 256 bytes están **reservados** para los **vectores de interrupción**, así que el código del programa debe estar colocado de la posición 256 en adelante, para evitar colisiones con los vectores.
- El primer vector de interrupción es muy importante ya que es el **vector de reset**. Aquí tiene que estar definida la dirección de inicio del programa. Más adelante se muestra con detalle en un ejemplo de código.

1.1.2. El adaptador de vídeo

De momento, el único dispositivo de E/S que tiene el sistema es **el adaptador de vídeo**. Se trata de un adaptador muy sencillo:

- Monocromo
- Modo texto
- 80 columnas, 60 filas

El formato del caracter es también muy sencillo: **cada byte se interpreta como un carácter ASCII**. Tiene una memoria de 8KiB y está mapeado en el comienzo del espacio de E/S. Esto significa que va desde 0 hasta 8191 inclusive, y puede accederse con **IN/OUT**. Un ejemplo para pintar una 'a' en la esquina superior izquierda de la pantalla:

```
load #'a'  
out 0
```

Cada fila de caracteres de la pantalla está mapeada **consecutivamente** en memoria, pero tiene un **padding de 48 posiciones**. Como cada fila tiene **80 caracteres visibles y 48 ocultos**, el comienzo de cada fila está separada por 128 posiciones de memoria. Para pintar por ejemplo un caracter en la cuarta fila, décima columna (4, 10):

```
load #'a'  
out 128*4 + 10
```

Esto se puede simplificar muchísimo haciendo uso de una **macro**, las cuales se explican más adelante.

1.2. Directivas

Directiva	Descripción	Ejemplo	Comentario
.org <address>	Modifica el contador de ubicación a <address>	.org 0x0100	A partir de ahora se colocarán los elementos en 0x0100
.d8 <byte1>, <byte2> ...	Define valores de 1 byte en memoria de manera consecutiva.	.d8 0x3F, 25, 0b10001100, 2*100 + 3	No es necesario usar hexadecimal; puede indicarse en decimal o binario.
.ascii <string>	Define una cadena de caracteres ASCII.	.ascii "Hola mundo"	Define la cadena "Hola mundo", terminada en carácter nulo (valor 0)
.d16 <dbyte1>, <dbyte2> ...	Define palabras de 2 bytes consecutivamente.	.d16 0x01FF, 128*2, 65535	El formato es Little Endian.
.r8 <size>	Reserva sin inicializar un buffer de <size> bytes.	.r8 256	Se reservan 256 posiciones de memoria.
.r16 <size>	Reserva de buffer de palabras de 2 bytes de tamaño <size>.	.r16 64	Se reservan 64*2 bytes, 128 bytes.
.equ <key> <value>	Define un símbolo constante.	.equ BUFFER_SIZE 256	BUFFER_SIZE se sustituye por 256 en tiempo de ensamblado.

Cuadro 1.1: Directivas del ensamblador MASIC

Notas:

- Todos los valores numéricos se pueden poner en decimal (250), hexadecimal (0xF0) ó binario (0b11111110), y la extremidad de los datos es Little Endian.
- También existen las directivas **.macro** y **.endm**, pero es conveniente que se expliquen más adelante en el apartado de las macros.

1.3. Instrucciones

Direccionamiento	Ejemplo	Descripción	Tamaño (en bytes)
Directo	ADD addr	Sumar al acumulador el contenido de la dirección addr	3
Inmediato de 8 bits	ADD # imm	Sumar al acumulador el valor imm	2
Inmediato de 16 bits	STSP # imm	Guardar en el registro del puntero a pila el valor imm	3
Indirecto	ADD [addr]	Sumar al acumulador el contenido apuntado por la dirección almacenada en addr	3
Sin operando	NOT	Se complementan todos los bits del acumulador	1

Cuadro 1.2: Distintos tipos de direccionamiento

Nota: el tamaño del valor inmediato viene implícito en el código de operación. Por ejemplo: la instrucción “ADD #imm” siempre espera un valor inmediato de **8 bits**, porque el acumulador es de 8 bits. En cambio, “STSP #imm” siempre espera un valor de **16 bits** porque el registro del puntero a pila tiene un tamaño de 16 bits.

1.3.1. Códigos de operación por instrucción y modo de direccionamiento

A continuación se muestran todos los códigos de operación de todos los modos de direccionamiento de todas las instrucciones.

Mnemónica	Directo	Inm.	Ind.	S/OP
JMP	0x00		0x2A	
CLC				0x01
STC				0x02
CLH				0x03
STH				0x04
SUBC	0x05	0x06		
LOAD	0x08	0x09	0x0A	
STORE	0x0B		0x0C	
ADD	0x0D	0x0E		
ADC	0x0F	0x10		
SUB	0x11	0x12		
AND	0x13	0x14		
OR	0x15	0x16		
NOT				0x17
CMP	0x19	0x1A		
NOP				0x1B
JV	0x1C			
JNV	0x1D			
JZ	0x1E			
JNZ	0x1F			
JC	0x20			
JNC	0x21			
JS	0x22			
JNS	0x23			

Cuadro 1.3: Primera tabla de códigos de operación

Las filas marcadas con **???** son códigos de operación que no están asignadas a ninguna instrucción, y son equivalentes a NOP.

Mnemónica	Directo	Inm.	Ind.	S/OP
SHL				0x2B
SHRA				0x2C
SHR				0x2D
ROL				0x2E
ROR				0x2F
SCL				0x37
SCR				0x38
RCL				0x3A
RCR				0x3B
PUSH				0x31
POP				0x32
CALL	0x33			
RET				0x34
INT		0x35		
RETI				0x36
IN	0x3D		0x30	
OUT	0x3E		0x3C	
LDSPH				0x39
LDSPH				0x3F
STSP	0x07	0x18		
???	0x24	0x25	0x26	0x27
???	0x28	0x29		

Cuadro 1.4: Segunda tabla de códigos de operación

1.4. Etiquetas y comentarios

Las **etiquetas** se pueden definir al comienzo de una línea terminándolas con dos puntos (:), y los **comentarios** estarán delimitados comenzando por un punto y coma (;) y terminando la línea. Aquí un ejemplo de código:

```
.d16 inicio

.org 0x0100 ;los primeros 256 bytes están reservados

inicio: ;definir la etiqueta inicio
    load #0 ;inicializo el acumulador con el valor 0

bucle: ;defino la etiqueta bucle
    add #1 ;sumo 1 al acumulador
    cmp #100 ;comparo el acumulador con el valor 100
    jnz bucle ;si la diferencia no es cero, volver a "bucle"
```

En el trozo de código anterior se puede ver un ejemplo de un **bucle que se ejecuta 100 veces**. En primer lugar, se define el vector de reseteo haciendo uso de la etiqueta “inicio” con la dirección a la que saltará el procesador en el reset (nótese que la etiqueta está definida más abajo). Luego, con la directiva `.org 0x0100` indicamos al ensamblador que nos encontramos ahora en la posición 0x0100 de memoria (Las posiciones de memoria que hay entre el vector de reseteo y la que estamos ahora serán rellenadas con ceros). En la línea 5 podemos ver la primera definición de una **etiqueta**: tiene como nombre “inicio” y se sabe que es una etiqueta porque la línea finaliza con dos puntos (:). Después de la instrucción “load #0”, podemos ver que se ha definido la etiqueta “bucle” en la línea 8.

Al final del código observamos como se hace uso de la etiqueta “bucle” en la instrucción “jnz bucle”.

Capítulo 1. EMASIC: Pequeña guía para el lenguaje

También podemos hacer uso de etiqueta + desplazamiento, que será calculado en tiempo de ensamblado:

```
buffer: .rb 50 ;reservo una cadena de 50 bytes  
  
load buffer+5 ;hago referencia al byte en la posición 5
```

La parte alta/baja de una dirección puede ser accedida con el modificador “**.low**” ó “**.high**”. Por ejemplo:

```
;hago referencia a un dato con la etiqueta "dato", que  
;se encuentra ubicado en la dirección 0xABCD  
.org 0xABCD  
dato: .d8 95  
  
;cargo el valor alto de la dirección de "dato"  
;la instrucción sería equivalente a "load #0xAB"  
load #dato.high
```

1.5. Macros

Las macros son **herramientas muy potentes** para programar en ensamblador. Permiten reemplazar trozos de código voluminoso y engorroso y son un valioso mecanismo de abstracción sin **ningún impacto** en la ejecución en términos de **rendimiento**. A continuación se muestra un ejemplo de definición y uso de una macro:

```
.d16 inicio

.org 0x0100

;las macros se pueden definir donde sea, pero siempre
;antes de ser utilizadas

.macro SUMAR dir1, dir2, dir3
    push ;guardamos el acumulador

    load %dir1
    add %dir2
    store %dir3

    pop ;restauramos el acumulador
.endm

.org 0x0100

operando1:
.d8 100
operando2:
.d8 23
destino:
.d8 0

inicio:
    SUMAR operando1, operando2, destino
    jmp inicio
```

Nota: las macros no deben tener nombres de palabras reservadas como las mnemónicas de las instrucciones ni tampoco deben empezar por punto (.) ni terminar en dos puntos (:). En tal caso se lanzará un **error de ensamblado**.

Las macros pueden tener tantos parámetros como sea necesario, y los argumentos que toman estos parámetros en la llamada a la macro pueden ser **expresiones matemáticas que pueden incluir símbolos y etiquetas**. Por ejemplo:

```
.d16 inicio

.org 0x0100

.macro SUMARIMM i1, i2
    add #%i1
    add #%i2
.endm

.equ VALOR 0x0F

inicio:
    SUMARIMM 10*3 + VALOR<<2, 20+2
jmp inicio
```

1.5.1. Etiquetas locales dentro de una macro

Existen dos tipos de etiquetas en el contexto de las macros:

- Una **etiqueta local** es una etiqueta definida dentro de esa macro (la cual **no** puede ser referenciada desde fuera)
- Las **etiquetas globales** son aquellas que han sido **definidas** fuera de cualquier macro (en un contexto global) y **sí** pueden referenciarse desde cualquier lugar del programa, **incluyendo dentro de una macro cualquiera**.

Sin embargo, **hay que tener cuidado**: Si una etiqueta **local** tiene el **mismo nombre** que una **global**, cuando se haga uso de ella, tendrá preferencia la etiqueta **local**, y no se generará ningún error de ensamblado.

```
.d16 inicio

.macro MACRO_PRUEBA
    jmp inicio
    dato: .d8 10
    inicio: ; esta etiqueta es local, y ocultará la global
           ; en el contexto de esta macro
    load dato
    out 0
.endm

.org 0x0100
inicio:
    MACRO_PRUEBA
    jmp inicio
```

Sin embargo, si se hace referencia a una etiqueta que sólo se ha definido globalmente, **no hay ambigüedad**: se usará el valor de dicha etiqueta. Dicho esto, queda claro que **sí** pueden usarse **etiquetas locales o globales** siempre que se haga con cuidado.

1.6. Expresiones

El ensamblador permite calcular expresiones como se ha visto, y se pueden hacer las siguientes operaciones:

- Suma (+)
- Resta (-)
- Multiplicación (*)
- División (/)
- Desplazamientos («, »)
- AND (&)
- OR (|)
- XOR (^)¹

Se pueden usar símbolos definidos con `.equ` o nombres de etiquetas para calcular las expresiones si es posible. Un ejemplo más abajo:

```
.d16 inicio
    .macro MACR01 param
        MACR02 %param
    .endm

    ;Imaginemos que queremos usar el bit de mayor peso de un
    ;parámetro del que aún no se conoce su valor:
    .macro MACR02 param
        load #(%param>>7)&1
    .endm
inicio:
    MACR01 0xFB
    add #'0'
    out 0 ;se imprimirá el bit de mayor peso de 0xFB
    jmp inicio
```

¹Por si no se ve bien, es el símbolo del circumflejo

Capítulo 2

QTMASIC: Cómo usar el emulador

2.1. Cargar código fuente

Lo más fácil para comenzar a programar, es abrir el **bloc de notas** o cualquier otro editor de texto, y escribir el código ahí. Cuando esté listo, abriremos **QTMASIC** y desde la barra superior, cargamos el archivo haciendo click en **Archivo > cargar código fuente...**

Si existe algún **error en el código**, saldrá una ventana mostrando la línea y columna donde hemos cometido el error, pero si está todo correcto, el programa comenzará a **ejecutarse inmediatamente**. La ventana nos muestra la salida del adaptador de vídeo en tiempo real.

2.2. Cargar código ya ensamblado

Si es necesario, puede cargarse un archivo de código máquina directamente desde el emulador desde **Archivo > Cargar código máquina...**

2.3. El depurador

Para probar el modo paso a paso, vamos a **Depurador > paso a paso**. Una vez marcada la casilla, la CPU se pondrá en pausa y aparecerá una barra con la que podremos:

- Reanudar/pausar la ejecución
- Avanzar 1 ciclo de reloj
- Avanzar 1 instrucción completa
- Resetear la CPU y recargar el programa

Para poder ver mejor lo que está sucediendo, si vamos a **Depurador > Mostrar memoria y registros**, podremos **ver** con todo lujo de detalles y **modificar** cualquier posición de **memoria** en tiempo real, además de los valores de los **registros**: sólo hay que hacer click en la posición de memoria o hacer doble click en la entrada deseada en la tabla de registros, y luego escribir en hexadecimal el valor deseado.

Cabe destacar que en este visor de memoria y registros, podemos ver el contenido al que apunta PC (**el contador de programa**) marcado en rojo.

2.3.1. Configuración

Podemos escalar la ventana y modificar el **periodo de reloj de la CPU** desde el menú de configuración de manera muy sencilla.